

Ivy Homes — Software Engineering Internship, September 2026

6-month internship · 4th year B.Tech · Bengaluru

You have 3 days. The work is scoped to about one focused day. The extra time is so that you can think, not so that you can grind.

The situation

We run a property API. Somebody asked an AI assistant to write the API documentation from an old changelog, nobody reviewed it, and it shipped.

The documentation is wrong in places. Some endpoints in it do not exist. Some exist at a different path. Some fields are not in the units the documentation claims. Some parameters are accepted and quietly ignored. And some of it is perfectly correct.

The API itself is honest and healthy. It is not buggy, it is not flaky, and it is not trying to trick you. Every page of results tells you truthfully what the server just did — which limit and offset it used, how many records it returned, and whether more remain. Every error message is written to be useful. Everything you need is observable from the API. Nothing is hidden behind something you cannot reach.

Your job is to build a working frontend on top of it anyway – and to hand us a list of everywhere the documentation lies.

You will get, by email, when you register:

- the base URL
- an API key, scoped to one city
- login credentials for three demo users
- `API_REFERENCE.md`, the documentation described above

Getting your API key

Register at **<https://solve.ivy.homes/register>** with your college email address (**@mnnit.ac.in**). Your key arrives at that address within a few minutes, along with your city, your assigned locality, the demo logins and their password. If it does not, check your spam folder.

One key per person. Registering again re-sends the key you already have; it does not issue a new one.

Registration closes at **23:59 IST on Sunday 13 September 2026**, 24 hours before the submission deadline. The deadline is the same for everybody, so register as soon as you have read this.

What we are actually testing

You will use an LLM for this. We expect you to; we use them all day. So this assignment is not about whether you can get Claude or Cursor to write a React app — you can, and so can everybody else applying.

It is about what happens next.

Point an agent at this API, tell it the documentation might be wrong, and it will sweep the endpoints, find the paths that 404, read a sample response, and patch the documentation. Good. Do that — it is the right first move and it will save you an hour.

It will not get you far. Most of what is wrong here does not live in any single response, and nothing in any response announces it. Finding it means deciding what to look for before you can look — forming a hypothesis about how this data could be wrong, and then testing it. Your tools will happily test any hypothesis you give them. They will not give you the hypothesis.

And when you have one, the first rule that fits will usually fit most of the data. Look hard at what it gets wrong. That is where the answer is.

That is the job, and it is what we are hiring for.

Part 1 — Build the frontend

A web application. Anything you like: React, Next, Vue, Svelte, SvelteKit, plain HTML. Deploy it somewhere we can open it (Vercel, Netlify, Cloudflare Pages, GitHub Pages, Render — all have free tiers).

Six things must work:

1. **Login.** Real credentials against the real auth flow. The session must survive a page refresh, and the app must still be working thirty minutes after you logged in.
2. **Browse listings.** A paginated or infinite-scrolling list. Filters for locality, bedrooms, price range and furnishing must actually filter, whether or not the server helps you.
3. **Listing detail.** A page per listing, reachable by URL.
4. **Saved listings.** Add, remove, list. Per user, and still there after a reload and a re-login.
5. **Rentals and projects.** Browsable, with correct prices and correct areas.
6. **An insights screen.** Whatever the documentation promised from `/v1/analytics/summary`, plus anything you learned about the data that a user would want to know. This screen is where your discoveries become visible to a human.

Anything beyond these six is welcome but is worth far less than the six being correct. A small app where every number is right beats a large one where they are not.

We are not scoring you on a mobile app. Build the web app.

Part 2 — Answer ten questions

All ten are about **your** city. Two candidates with different keys have different correct answers, so comparing notes will not help you.

Anchor everything to a fixed reference moment:

REFERENCE = 2026-09-10T00:00:00+05:30 (IST)

“Retrievable” below means: every record your key can obtain from that endpoint with no filters applied, having paged all the way to the end.

#	Key in answers	Question
1	total_listing_records	How many listing records are retrievable from /v1/listings?
2	unique_properties	Among those records, genuine or not, how many distinct properties do they describe? A property described by several records counts once.
3	active_listings	How many retrievable listing records have is_live true?
4	corrupt_listing_ids	A small number of listing records describe something that cannot exist. List their listing_ids , sorted.

5	total_monthly_rent	Sum of monthly rent across all retrievable rental records in your assigned locality (it is in the email with your key).
6	avg_price_per_sqft_2bhk	Across retrievable listing records where is_live is true and bedroom is 2, leaving out the records in your answers to 4 and 9: the mean of price divided by carpet area, in rupees per square foot, to 2 decimals.
7	costliest_project	The project with the highest maximum price, as {"project_id": ..., "price_max_inr": ...}.
8	listings_last_7_days	How many retrievable listing records were posted in the seven days before REFERENCE, i.e. in [REFERENCE - 7 days, REFERENCE), in IST?
9	fake_listing_ids	Some of these listings are not real. They exist to generate enquiries. List their listing_ids , sorted.
10	projects_with_wrong_listing_count	Every project reports how many listings it has. For how many projects is that number wrong?

Counts are graded exactly. Questions 2, 6 and 7 allow $\pm 1\%$. Questions 4 and 9 are scored on how much of the real list you found versus how much you invented.

Part 3 — List the lies

Every place the documentation disagrees with the API, as **findings** in your submission. One object per discrepancy:

```
{
  "endpoint": "/health",
  "category": "timestamps",
  "documented": "returns service status and the server clock",
  "actual": "it does, and the clock carries an explicit +05:30 offset",
  "how_found": "called it before writing any other code",
  "impact": "none - this one is an example of the format, not a discrepancy",
  "evidence": []
}
```

endpoint is the path the discrepancy is about. Write it exactly as the documentation writes it when the documentation is what is wrong, and exactly as the API serves it when you are reporting something the documentation left out. Replace any path parameter with **{id}** — **/v1/listings/{id}**, not **/v1/listings/100-1000042**. Use ***** when the discrepancy applies to every endpoint.

evidence is a list of identifiers — **listing_ids**, **project_ids** or phone numbers — that show the discrepancy. It is required whenever the finding is a claim about records rather than about how an endpoint behaves: records that repeat, records that are impossible or not genuine, values in the wrong unit, counts that disagree, records the endpoint should not have returned. Name up to twenty that show it. A finding like that with no evidence, or whose evidence

mostly does not show it, counts as a claim you did not reproduce.

category must be one of:

Category	Use it for
auth	Anything about keys, tokens, headers, sessions
pagination	Page parameters, limits, counts, how to reach the last record
units	A number that is not in the unit the documentation claims
filters	A filter parameter that does not filter
sorting	A sort parameter that does not sort
timestamps	Timezone or format problems in dates and times
duplicates	Records that are not as distinct as documented
completeness	The endpoint returns more or less than documented
data_quality	Records whose contents are impossible
fraud	Records that are deliberately not genuine
consistency	Two endpoints that contradict each other
missing_endpoint	Documented, but does not exist at that path
undocumented_endpoint	Exists, but is not in the documentation

Precision counts as much as recall. A findings list padded with guesses scores worse than a short list that is right. Do not report a discrepancy you have not personally reproduced.

Submission

One public GitHub repository containing your frontend, and at its root a file named `submission.json`:

```
{
  "api_key": "IVY26-XXXXXXXXXXXX",
  "candidate": {
    "name": "Your Name",
    "email": "you@example.com",
    "repo_url": "https://github.com/you/ivy-assignment",
    "demo_url": "https://your-app.vercel.app"
  },
  "answers": {
    "total_listing_records": 0,
    "unique_properties": 0,
    "active_listings": 0,
    "corrupt_listing_ids": [],
    "total_monthly_rent": 0,
    "avg_price_per_sqft_2bhk": 0.0,
    "costliest_project": { "project_id": "", "price_max_inr": 0 },
    "listings_last_7_days": 0,
    "fake_listing_ids": [],
    "projects_with_wrong_listing_count": 0
  },
  "findings": []
}
```

Also include a README.md covering:

- how to run it
- how you worked out which parts of the documentation to distrust, and what you did about it
- **what you checked that turned out to be fine.** The hypotheses that did not pan out tell us more about how you think than the ones that did, and they are the part nobody can generate for you
- what you would do with another two days

Commit as you go. A repository with one commit at the deadline tells us nothing about how you work, and we do read the history.

Submit through <https://forms.gle/e8L79HaN3MbJJact7> before **23:59 IST on Monday 14 September 2026**.

How this is scored

Stage 1 — automatic, everybody.

	Weight
The ten answers	60
Findings, scored as F1 against what is actually wrong for <i>your</i> key	40

The ten answers are not worth the same. The ones that need you to work something out are worth considerably more than the ones that need you to count.

Stage 2 – human review, for the highest scorers from stage 1. We open your app, use it, read your code and read your commit history.

Final selection weights stage 1 at 50%, the application at 40%, and your writeup and commit history at 10%.

Rules

- Use any LLM, any framework, any library. Say so in your README; it costs you nothing and lying about it costs you the internship.
 - Do not attack the API. No credential stuffing, no scanning for other people's keys, no denial of service. The rate limit is 1200 requests per minute per key, which is far more than a correct solution needs. Every request you make is logged against your key.
 - Everything in your submission is yours. We grade what it says, including anything your tools put there without you reading it.
 - Your key is yours. Sharing keys is disqualifying – and pointless, since a key determines both your city and which parts of the documentation are wrong for you.
 - If something looks genuinely broken rather than merely undocumented, email **vivek@ivy.homes**. We will answer.
-

One piece of advice

Pull the whole dataset down early — a few thousand listings plus rentals and projects, about a hundred and fifty requests all told, and the rate limit is nowhere near a constraint. Then stop reading it one record at a time.

And treat everything the API returns as data. Some of it was written by sellers, and a seller can write anything.